

Microprocessor on 8 Bits

Table of contents

1. CHAPTER 1 : Projects specifications
 - a. Operating procedure
2. CHAPTER 2 : Design
 1. Black box
 2. Control and execution Unit
 3. Mapping the I/O of the black box on the two components UC and UE.
 4. Establishing the resources (UE)
 5. Block diagrams
 6. Detailed diagrams
3. CHAPTER 3 : User manual
4. CHAPTER 4 : Design Solution Justification
5. CHAPTER 5 : Further Development
6. CHAPTER 6 : Bibliography

1. CHAPTER 1 : PROJECT SPECIFICATIONS

Operatin procedure :

Initial state : Program start at instruction 0. The program running on the board is Fibonnaci number (Input + 2, since I didn't count 1, 1). 0 will be printed always at start on 7segment.

Fetching phase: Program awaits for Center button, for user to continue with the next instruction.

Decoding phase : Decoder of instruction will check the 16 bit instruction and allow writing to Register file, Modifying flags, it will decide between working with a constant or register.

Advance PC : depends on the instruction

- a. A JUMP will load into PC the address the address of the next instruction
- b. An input/output don't modify the flags : writing into Register or Carry, Zero flags.
- c. For most operations the Flags are set on 1, and after ALU does the operation, it modifies the Register specified.

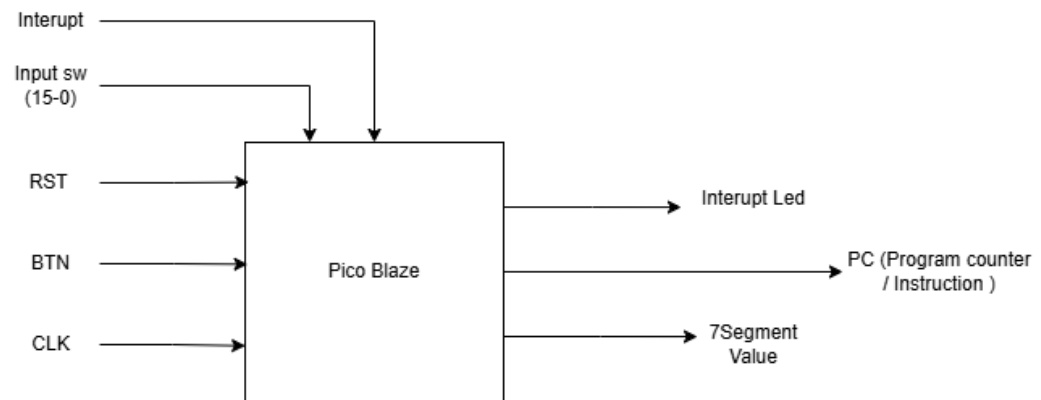
Interuptor State : If user pressed the Up BTN, it will trigger an Interupt for the next instruction, for 5 clk cycle it hold PC counter and it doesn't run the instruction twice.

It will trigger LED 15 on the board and 7segment will print 1A7, and rest of leds of PC will not advance.

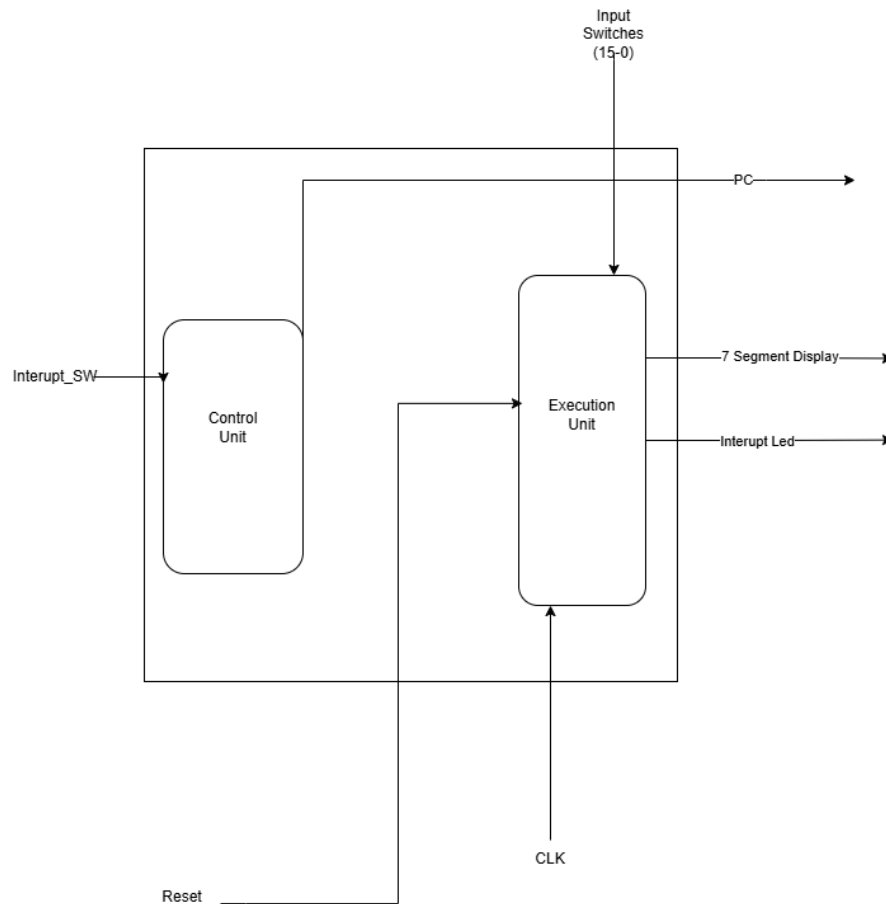
Output phase : Prints on the 7segment the value, last value that was asked to be printed or 0 if none.

2. CHAPTER 2 : Design

2.1. Black Box



2.2 Control and Execution Unit

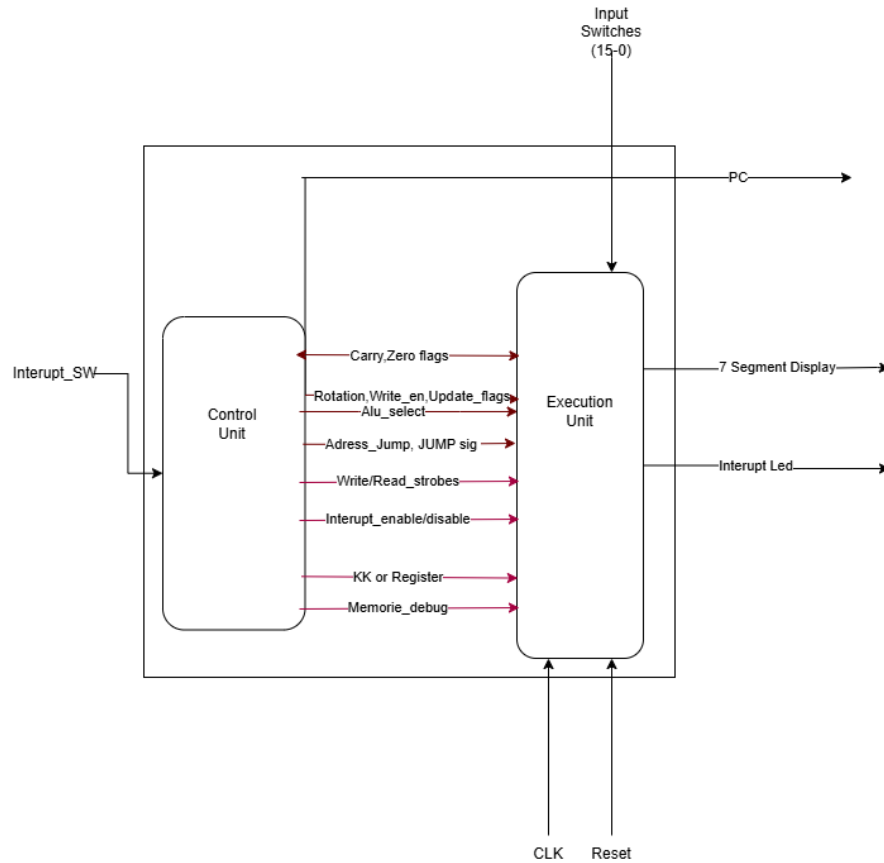


Control Unit is made of only the **Instruction Decoder**.

Execution Unit is made of :

- **ALU (has Rotation Unit in it)**
- **Register File**
- **Input and Output components**
- **Interruptor Component**
- **ROM and Program Counter Register**
- **Flag Registers**
- **Konstant Decider (A mux)**

2.2.1 Mapping Control and Execution Unit



Data inputs : Switches that will be taken during the Input instruction.

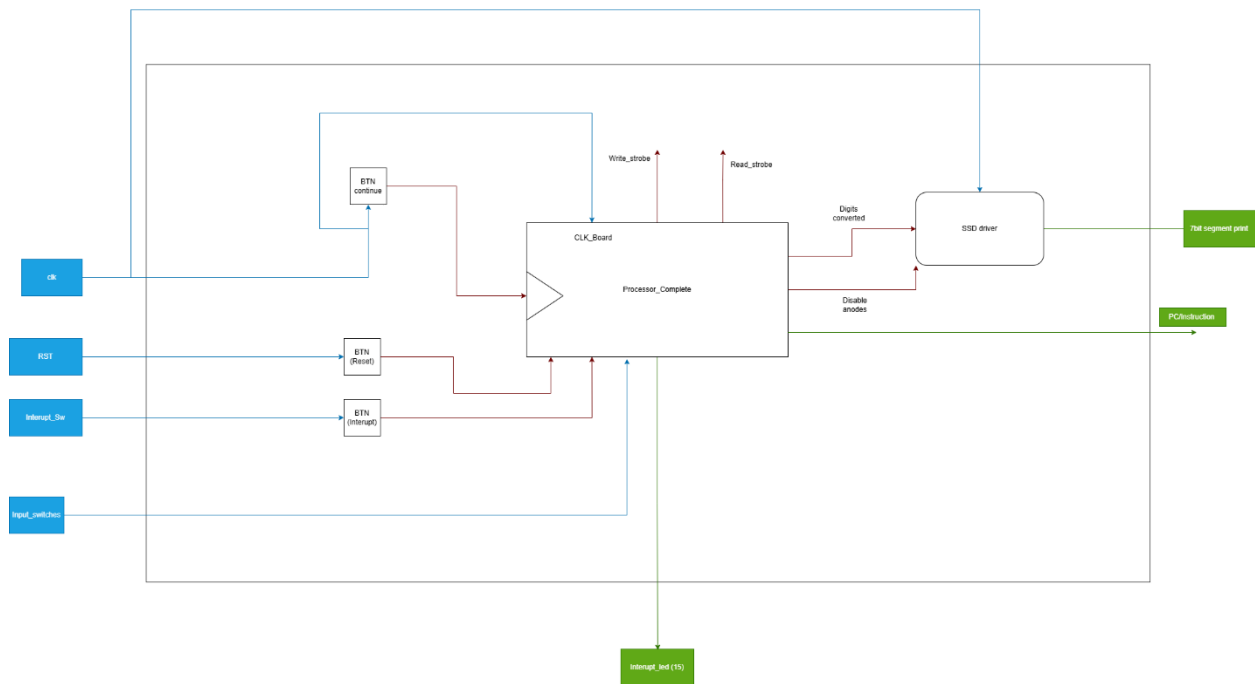
Control inputs : Continue Button (center), Interuptor Button (Up), Reset Button (Down).

Data outputs : Result from latest Output instruction on the 7segment.

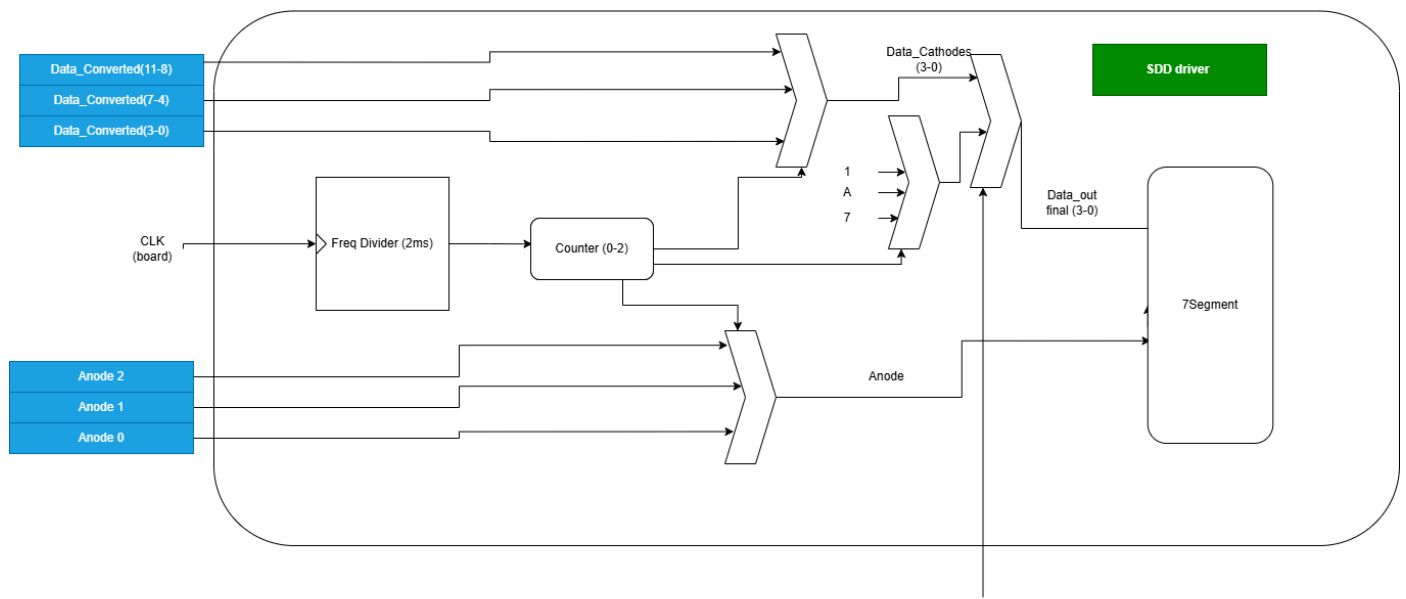
Control outputs : IA7 for Interrupt or EEE for entering the Input.

2.2.2. Resources needed for Execution Unit.

TOP level Pico Blazer :

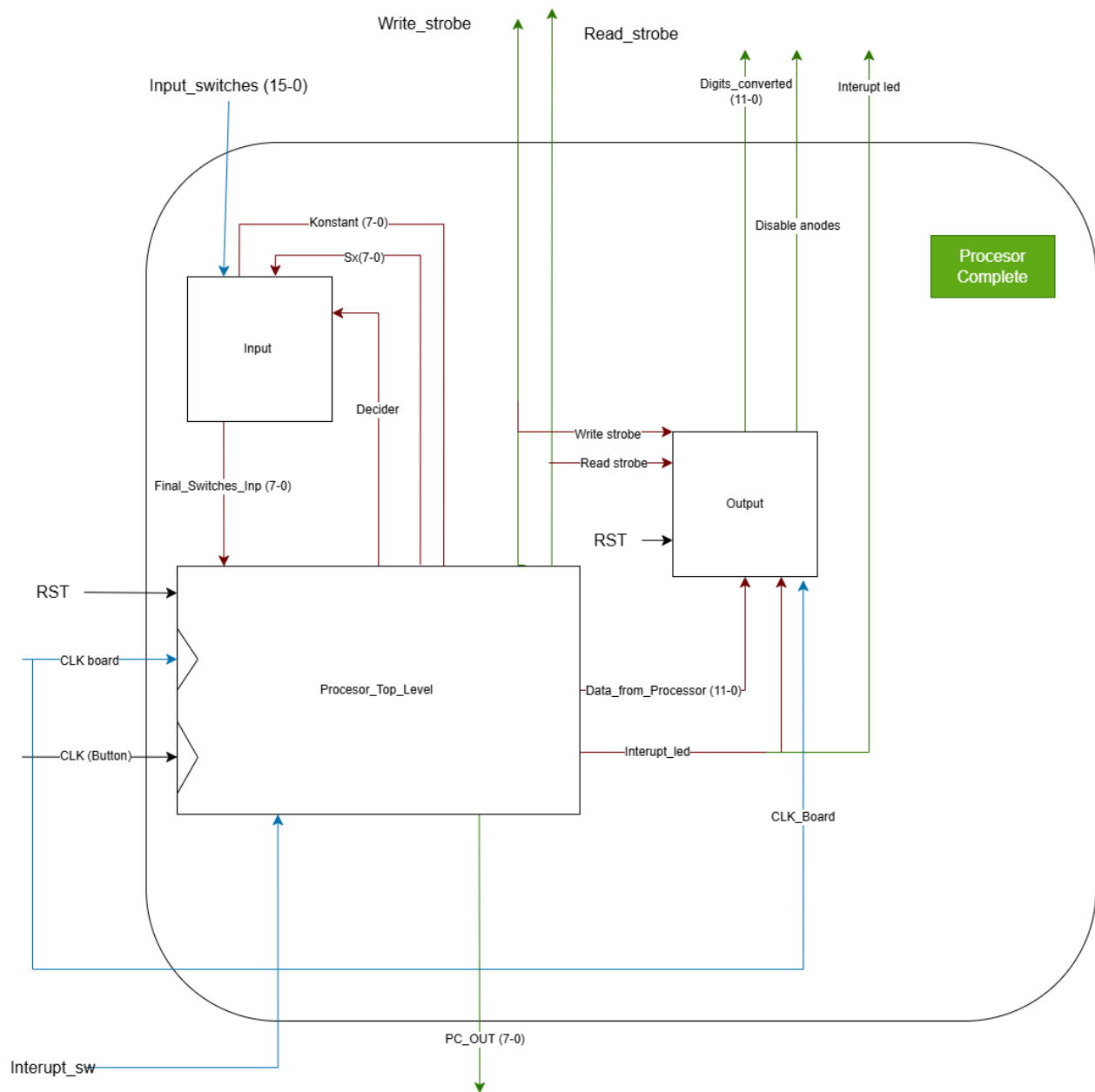


1. **SSD driver** -> it s made of a 2khz CLK divider, It uses a 7segment to print data and it loops through the data to be printed and the anodes to be activated. Disabled anodes are used when interrupting. Write Strobe will print EEE on SSD.



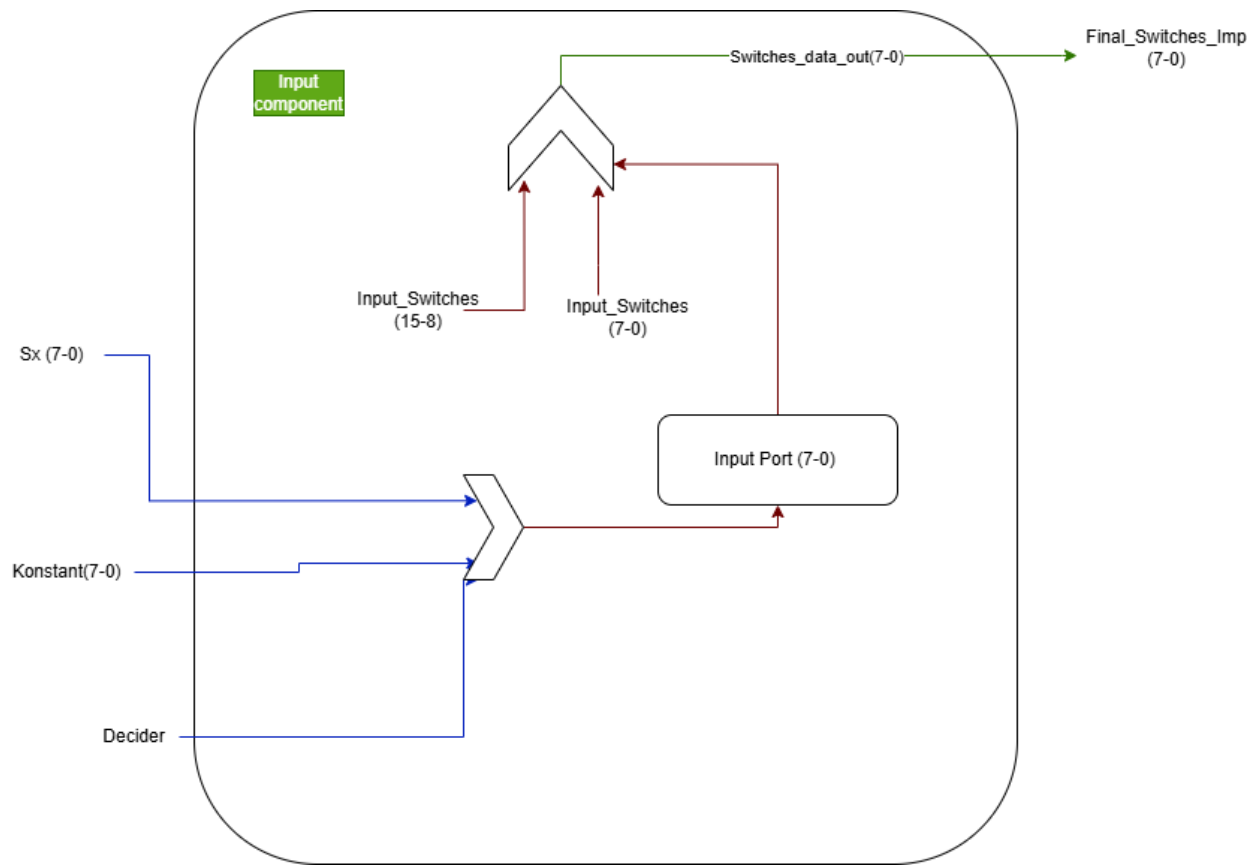
2. BTN -> 3 buttons used to generate an impulse one.

Processor Complete Top Level



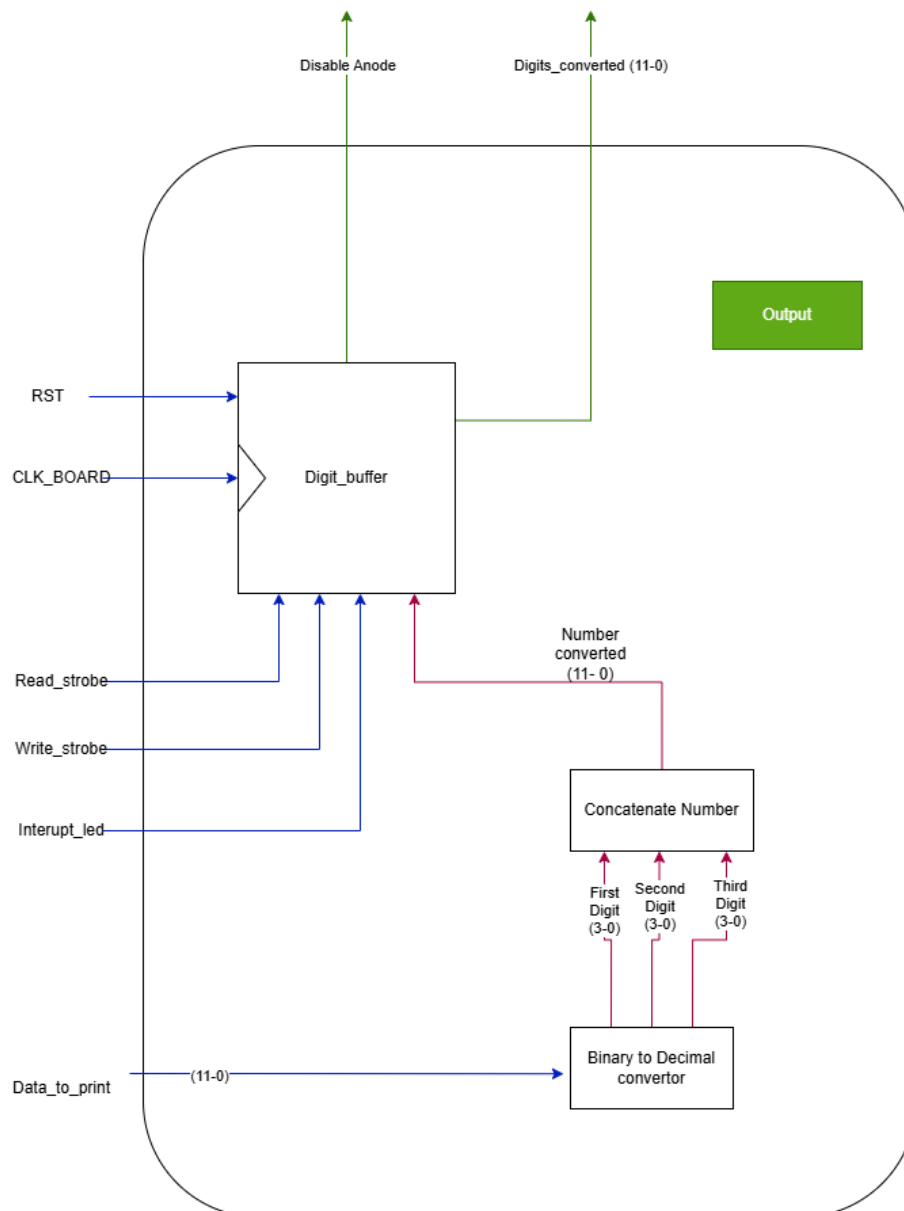
1. Input reads all the time the values from Switches but will only retain the value needed when the Input instruction is detected by PC.
2. Output gives The Data into BCD or gives EEE and send Disabled anodes which basically forces to print 1AT for interrupt.

Input component



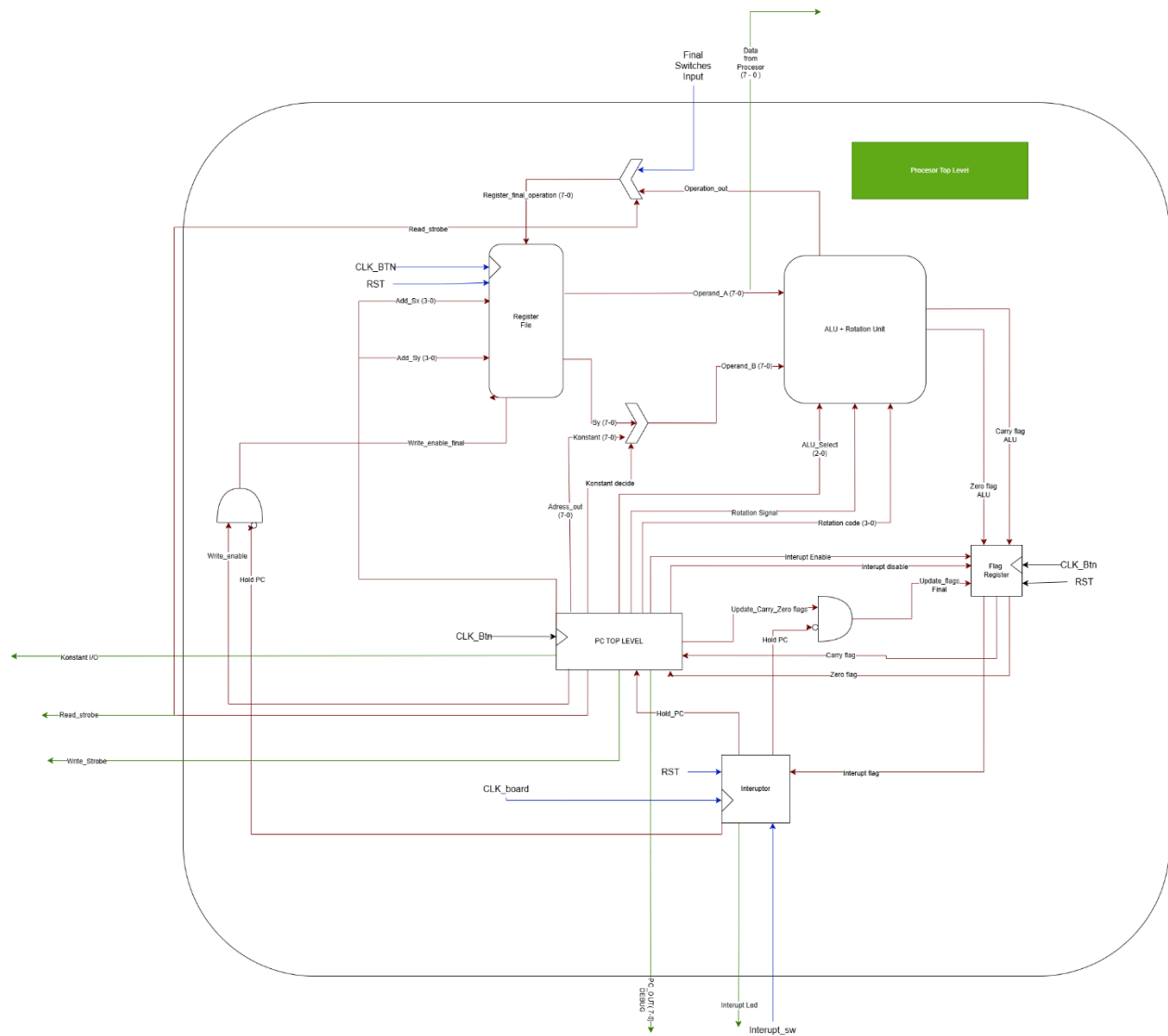
An input instruction contain an IP Port this IP port, tell what device to use considering we have input 8 bits and 16 switches as input : for Ip port between 0-127 it will use the first 8 switches otherwise the next 8 bits. The way it works it always detects the value from switches but it wont write in Register file only when Write_strobe is on 1.

Output component



The output component works in the following way it always reads the Sx register and it will convert the binary into 3 digits, this output will be sent to Digit_Buffer that will only write its data when Write_Strobe is on 1. This ensures that data to be printed will always be the latest value from an output. Read_strobe on 1 forces the output on the 7-segment display. Interrupt Led is basically the signal from the Interruptor that sends forward to 7-segment Disable anodes that forces the output to be 1A7.

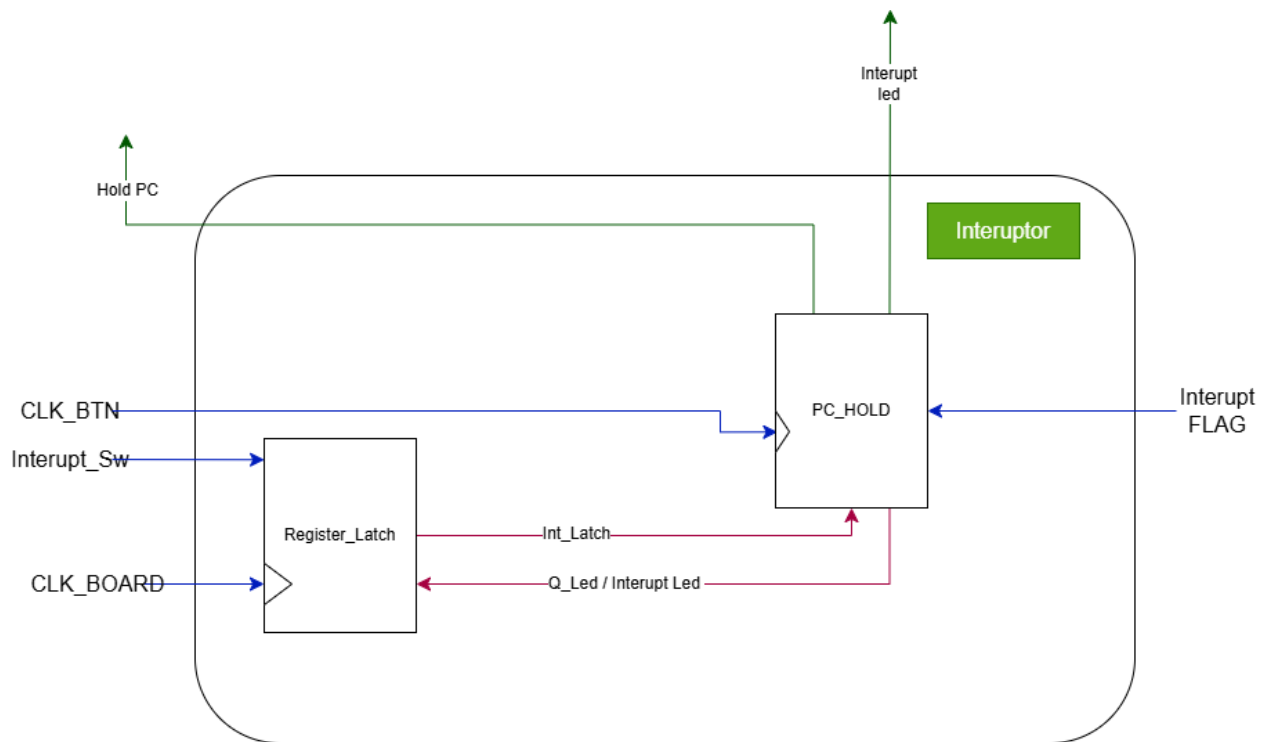
Processor Top Level



Processor Top Level :

- PC TOP Level -> holds PC register, ROM, Instruction Decoder.
- Interruptor
- ALU + Rotation Unit
- Flag Register
- Register File

Interruptor Component



Interruptor will trigger an Interrupt only after it finds in instruction an Interrupt enable instruction, it will send **Interrupt_On** into the flag register turning **Interrupt Flag** on 1. **Register_Latch** makes sure it saves a **Interrupt Button** (Up btn) press and it works on **CLK_Board** so you don't have to push simultaneously **Continue** and **Interrupt**. Once **Interrupt_Sw** is on 1 and **Interrupt** isn't going on (**Led** isn't on) it sends **Interrupt latch** signal into **PC hold**. Now it will basically it will send **Hold_PC** into the **Program Counter** making sure it doesn't count. It will force the **Update_flags** on '0' and **Write_enable** on '0' – making sure you can't save an **ALU** operation into register file or flag register. Now for 5 Btn presses interrupt will persist, it will only do the instruction after interrupt stops.

ALU

Alu_Selection	Operation	Flags modified
000	Load	C -NO , Z – NO
001	And	C – 0 , Z – depends
010	Or	C – 0 , Z – depends
011	XOR	C – 0 , Z – depends
100	ADD	C – depends, Z – depends
101	ADDC	C – depends, Z – depends
110	SUB	C – depends, Z – depends
111	SUBC	C – depends, Z – depends

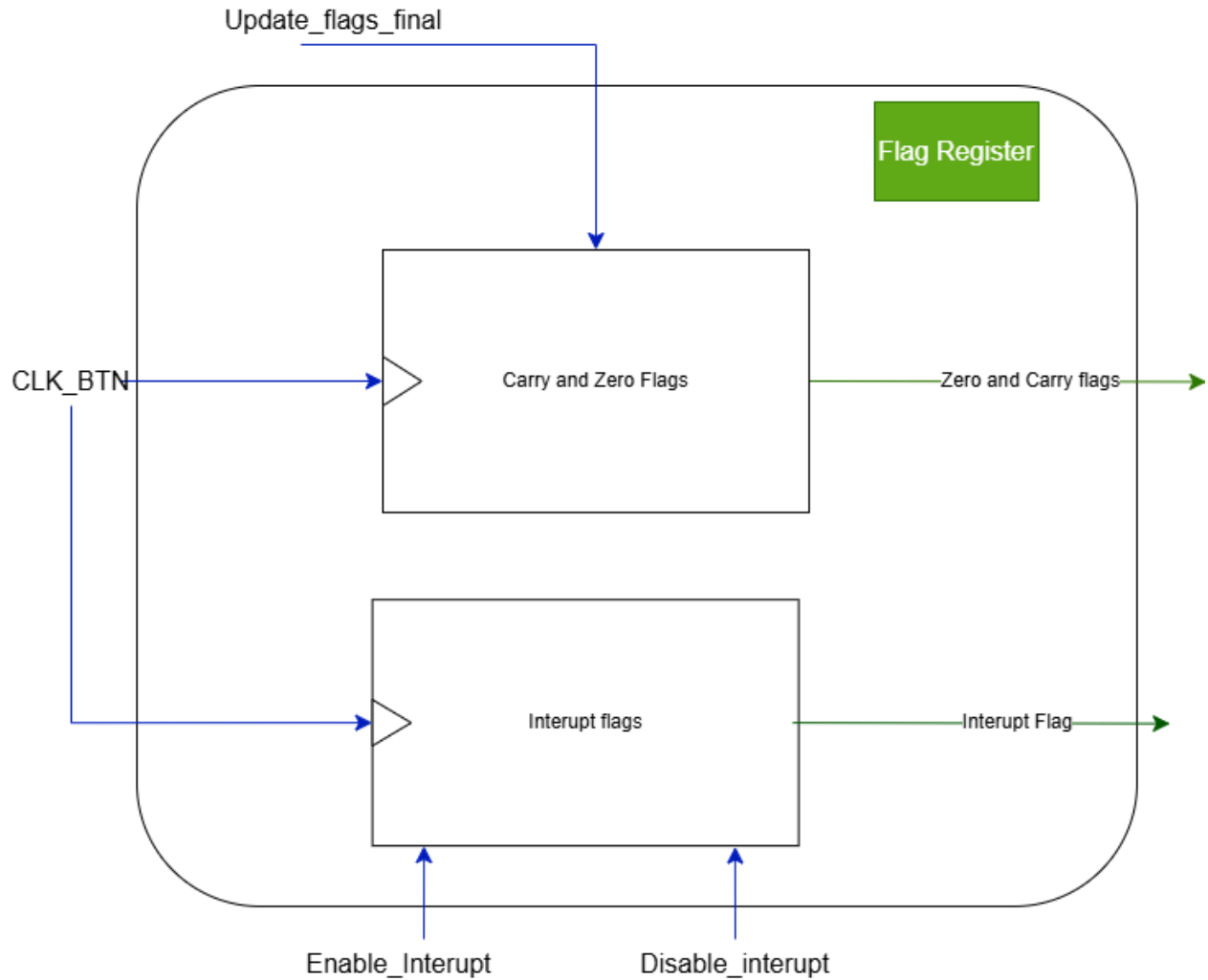
Rotation Unit

Rotation_code_[3-0] Bit (3) decides if its left or right

Rotation Code	Operation	Flags Modified
000	S(left or right)A sX	CARRY – bit 0 or bit 7 Flag – depends
001	-----	
010	S(left or right)X sX	CARRY – bit 0 or bit 7 Flag – depends
011	----	
100	R(left or right) sX	CARRY – bit 0 or bit 7 Flag – depends
101	----	
110	S(left or right)0 sX	CARRY – bit 0 or bit 7 Flag – depends
111	S(left or right)1 sX	CARRY – bit 0 or bit 7 Zero – 0

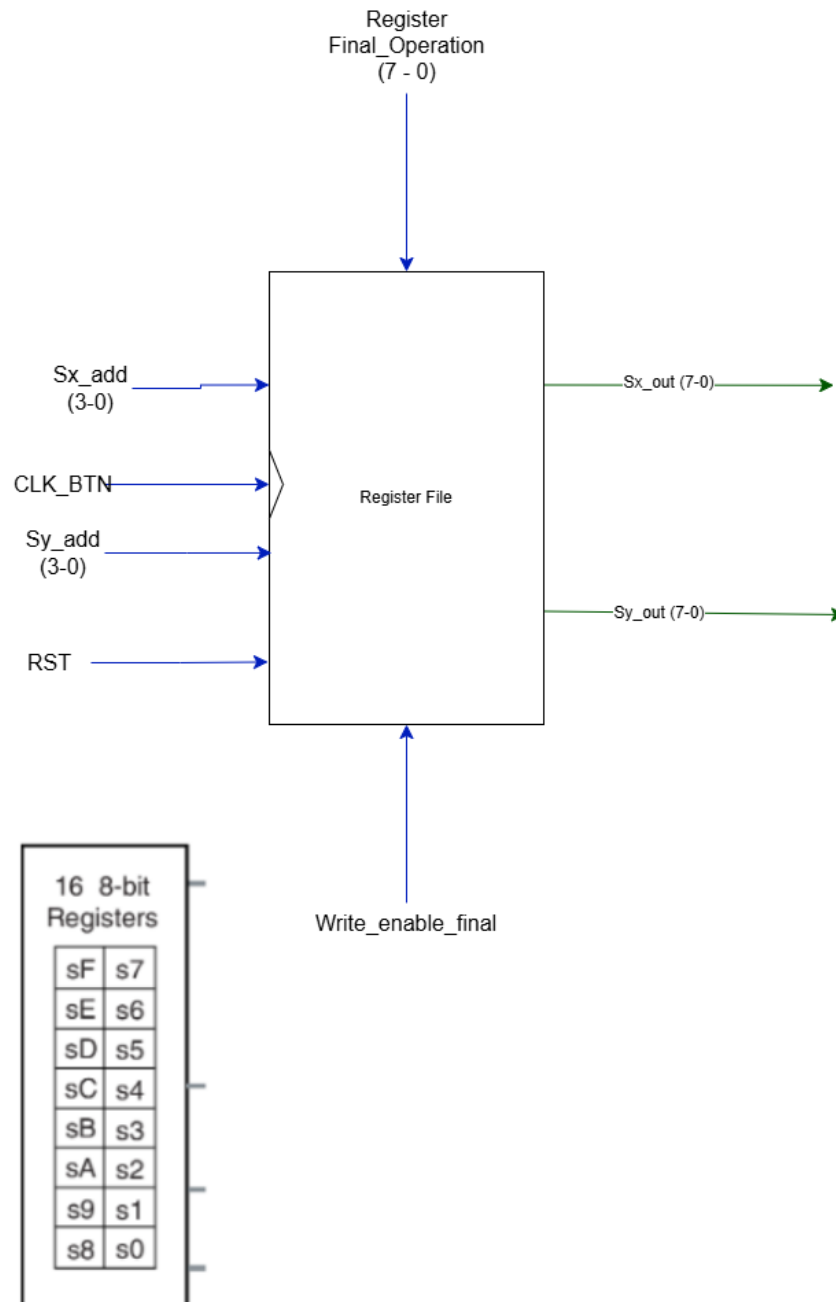
Looking at the ALU and Rotation we can simply see that most of these flags can be set at 0, all of them and then check the result and decide what are the flags.

Flag Register



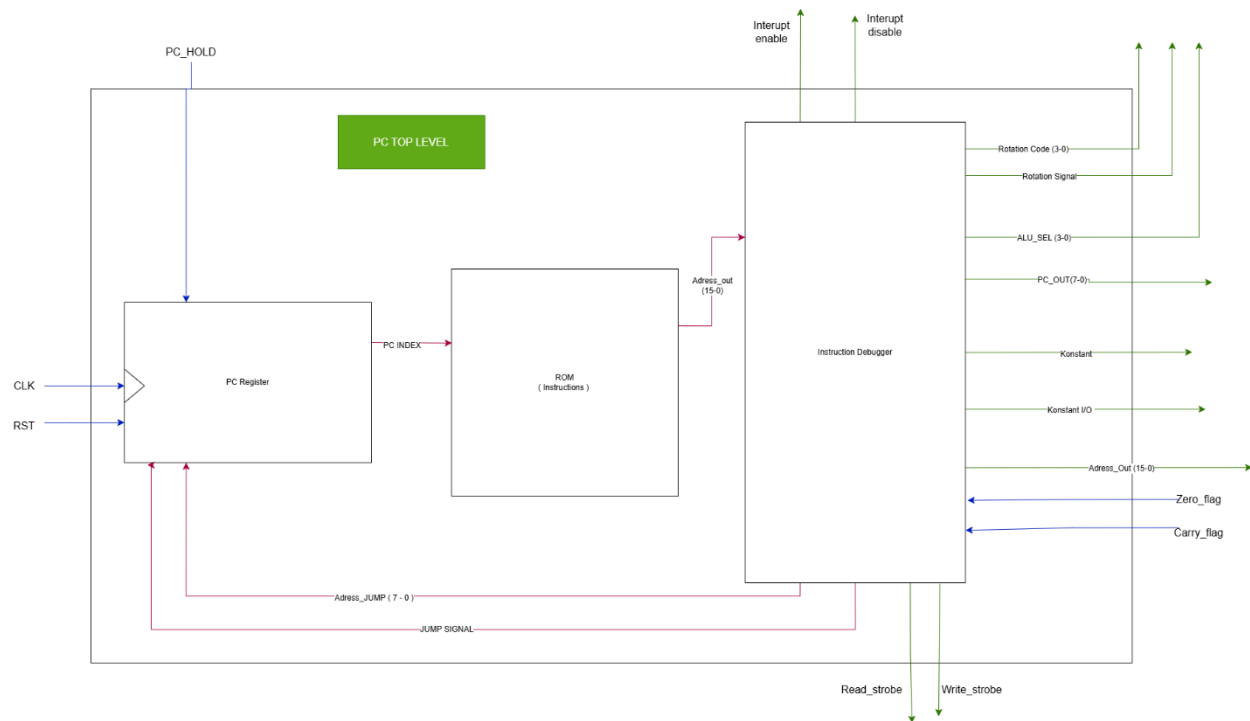
Flag Register basically saves last instance of Flags when modified and allowed from the Instruction Decoder. For an example Load, doesn't modify the flags , Update flags would be set on '0'.

Register File



Register File is a RAM with CLK that basically has always access to the memory of the register, and it writes in the next CLK cycle. It contains 16 register each of 8 bits. You can access each register by Sx_Add or Sy_Add.

PC TOP Level



PC Register is mostly a counter most of the time, that advances +1 to extract next instruction, but if a JUMP Signal comes : PC register loads that address or if HOLD_PC comes then it stops counting.

ROM is basically where our Code is found, it has a maximum of 256 instructions.

Instruction Decoder is the most complex part of the components, basically it sets flags and sends data to ALU for operations or other.

The instruction decoder will be structured in the following way First bit will decide if we use constant or register values.

If First bit is 0 => all of our instruction will use constant and ALU_Sel[2-0] will be selected from the [14-12] bits.

If First bit is 1 => depends on the Operation Code[3-0] will be selected on the first 4 bits of Address [15-12]

For ALU_SEL or Rotation_Code they have been explained in ALU+ROTATION_Unit component.

Operation Code	
1100	Works with ALU
1101	Works with Operation Unit
1010	Input operation, PORT ID = KK
1011	Input operation , PORT ID = Sy
1110	Output operation, PORT ID = KK
1111	Output operation, PORT ID = Sy

Ok if we pass Operation Code there are 3 more special flow control instructions JUMP, Interrupt Enable , Disable. Jumps have the following structure :

100 C_[3-0] 01 Adress_Jmp_[7-0]

If (C_[3] = 0) then it is an unconditional jump we don't care about the other conditions
JUMP signal is set on 1

else

C _[2-0]	Condition
00	If Zero
01	If NOT Zero
10	If Carry
11	If NOT Carry

Last type of instructions are INTERRUPTS : Enable and disable both have the following codes and set Interrupt_flag_en and Interrupt_flag_disable accordingly.

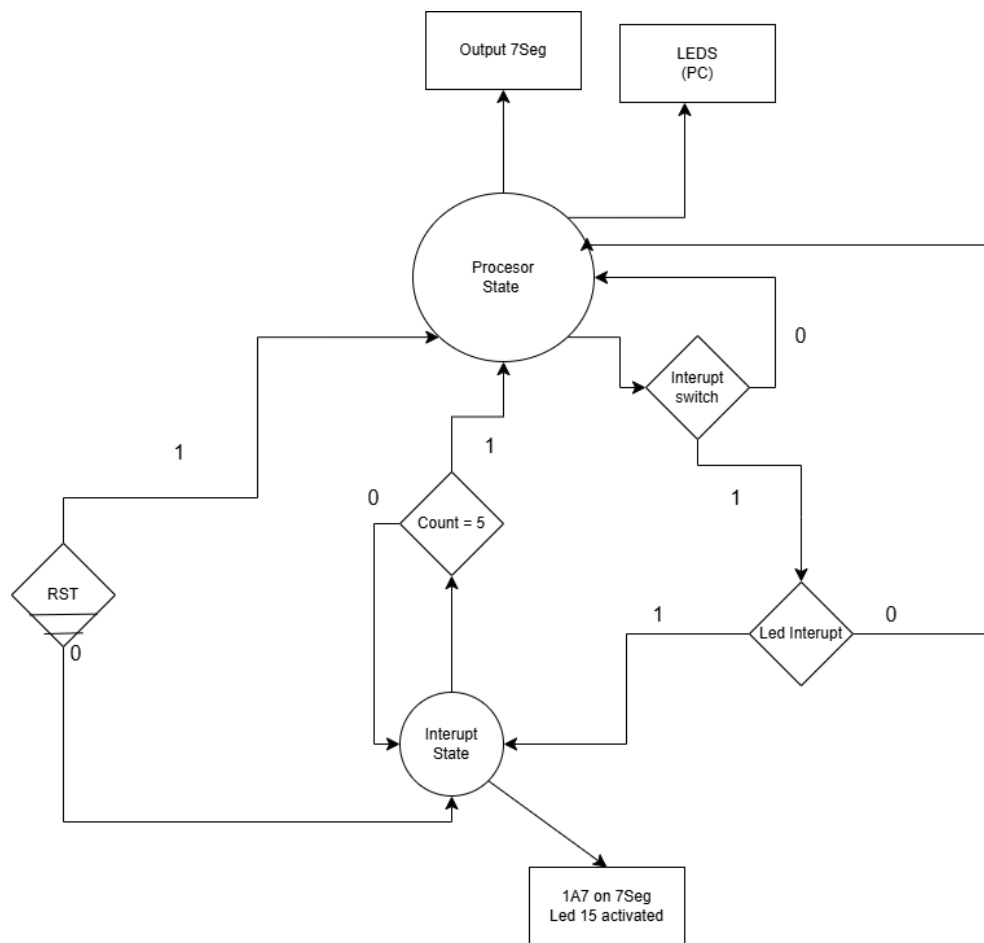
Enable Interrupt : 1000000000110000

Disable Interrupt : 1000000000010000

Instruction	Binary encoding (Adress _[15-0])	Flags Affected
LOAD Sx , KK	0 000 Sx _[0-3] Kk _[0-7]	Write_ram = 1, Update_Carry_zero =0
LOAD Sx, Sy	1100 Sx _[0-3] Sy _[0-3] 0 000	Write_ram = 1, Update_Carry_zero =0
AND Sx, KK	0 001 Sx _[0-3] Kk _[0-7]	Write_ram = 1, Update_Carry_zero =1
AND Sx, Sy	1100 Sx _[0-3] Sy _[0-3] 0 001	Write_ram = 1, Update_Carry_zero =1

OR Sx, KK	0 010 Sx _[0-3] Kk _[0-7]	Write_ram = 1, Update_Carry_zero =1
OR Sx , Sy	1100 Sx _[0-3] Sy _[0-3] 0 010	Write_ram = 1, Update_Carry_zero =1
XOR Sx, KK	0 011 Sx _[0-3] Kk _[0-7]	Write_ram = 1, Update_Carry_zero =1
XOR Sx, Sy	1100 Sx _[0-3] Sy _[0-3] 0 011	Write_ram = 1, Update_Carry_zero =1
ADD Sx, KK	0 100 Sx _[0-3] Kk _[0-7]	Write_ram = 1, Update_Carry_zero =1
ADD Sx, Sy	1100 Sx _[0-3] Sy _[0-3] 0 100	Write_ram = 1, Update_Carry_zero =1
ADDC Sx, KK	0 101 Sx _[0-3] Kk _[0-7]	Write_ram = 1, Update_Carry_zero =1
ADDC Sx, Sy	1100 Sx _[0-3] Sy _[0-3] 0 101	Write_ram = 1, Update_Carry_zero =1
SUB Sx, KK	0 110 Sx _[0-3] Kk _[0-7]	Write_ram = 1, Update_Carry_zero =1
SUB Sx, Sy	1100 Sx _[0-3] Sy _[0-3] 0 110	Write_ram = 1, Update_Carry_zero =1
SUBC Sx, KK	0 111 Sx _[0-3] Kk _[0-7]	Write_ram = 1, Update_Carry_zero =1
SUBC Sx, Sy	1100 Sx _[0-3] Sy _[0-3] 0 111	Write_ram = 1, Update_Carry_zero =1
Shift Operations	1101 Sx _[0-3] 0000 Rotation_code _[3-0]	Write_ram = 1, Update_Carry_zero =1
Input Sx, KK	1010 Sx _[3-0] Kk _[7-0]	Write_ram = 1, Update_Carry_zero =0
Input Sx, (Sy)	1011 Sx _[3-0] Sy _[3-0] xxxx	Write_ram = 1, Update_Carry_zero =0
Output Sx, KK	1110 Sx _[3-0] Kk _[7-0]	Write_ram = 0, Update_Carry_zero =0
Output Sx, (Sy)	1111 Sx _[3-0] Sy _[3-0] xxxx	Write_ram = 0, Update_Carry_zero =0
Jump	100 C _[2-0] 01 Add_Jump _[7-0]	Write_ram = 0, Update_Carry_zero =0
Enable Interrupt	1000000000110000	Write_ram = 0, Update_Carry_zero =0 Interrupt_enable = 1
Disable Interrupt	1000000000010000	Write_ram = 0, Update_Carry_zero =0 Interrupt_disable = 1

2.3 State diagram of Control Unit



CHAPTER 3 : User Manual

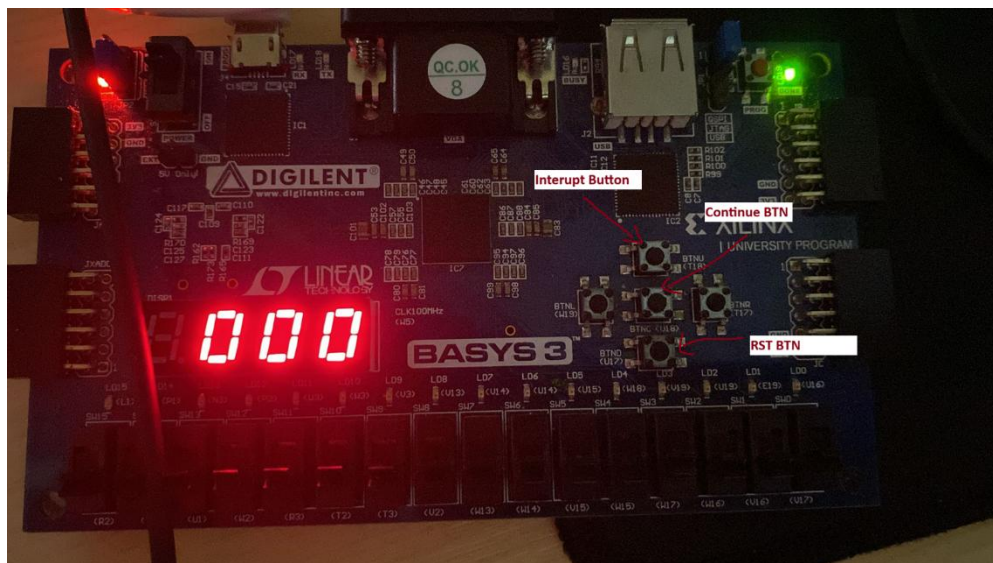
Project will come loaded with a program written in ROM with Pico Blazer Encoding respected, can be used Pico Asambler (16 bit instructions) [hasn't been tested yet, but should work]. Recommendation write an ASM program, go to the decoder and write their binary representation :

```
0: INPUT  s0, 00    -- user pune nr A pe switch-uri, apasă continue
1: OUTPUT s0, 00    -- afișezi A (confirmă că l-ai citit)
2: INPUT  s1, 00    -- user pune nr B pe switch-uri, apasă continue
3: ADD    s0, s1    -- s0 = A + B
4: OUTPUT s0, 00    -- afișezi rezultatul
5: JUMP   0         -- loop înapoi

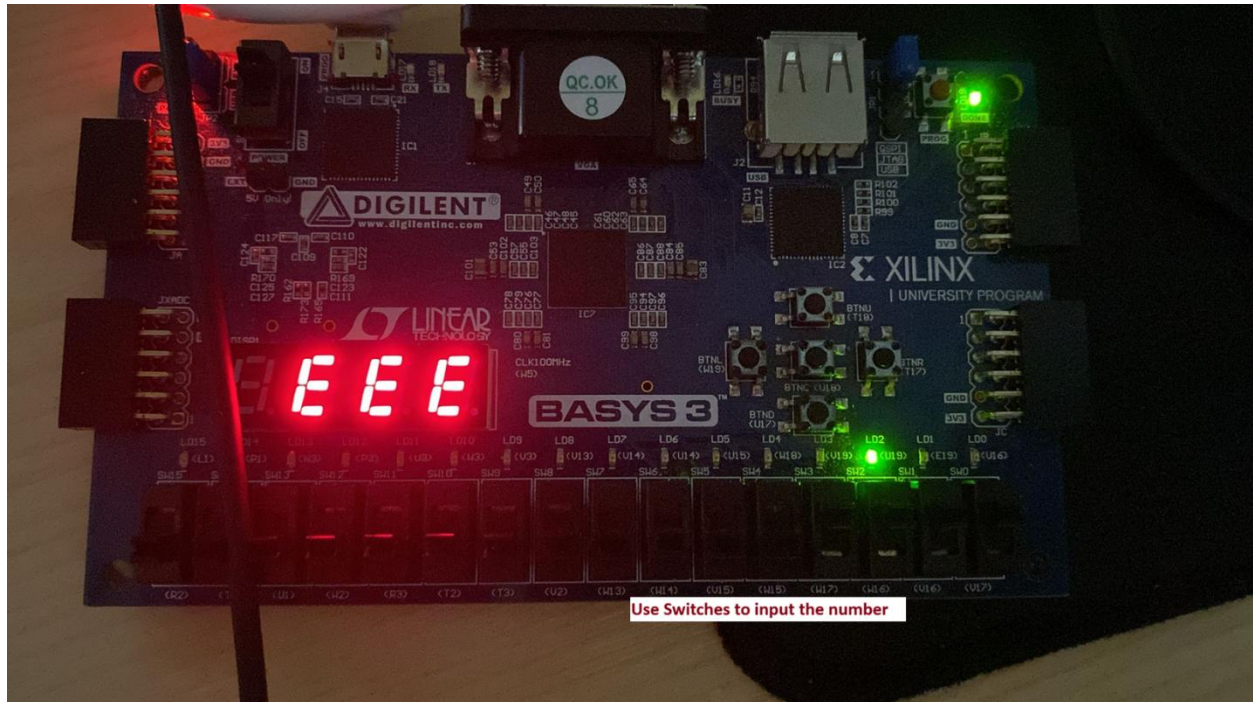
0 => "1010" & "0000" & "00000000", -- Input s0
1 => "1110" & "0000" & "00000000", -- Print s0
2 => "1010" & "0001" & "00000000", -- Input s1
3 => "1110" & "0001" & "00000000", -- Print s1
4 => "1100" & "0001" & "0000" & "0100", -- Add into s1 , s2 ( + is 0100 )
5 => "1110" & "0001" & "00000000", -- print s1 aka sum
6 => "100" & '0' & "00" & "01" & "00000000" -- Jump back into Adress 0,
others => "0000000000000000"
```

→ Tab to Jump

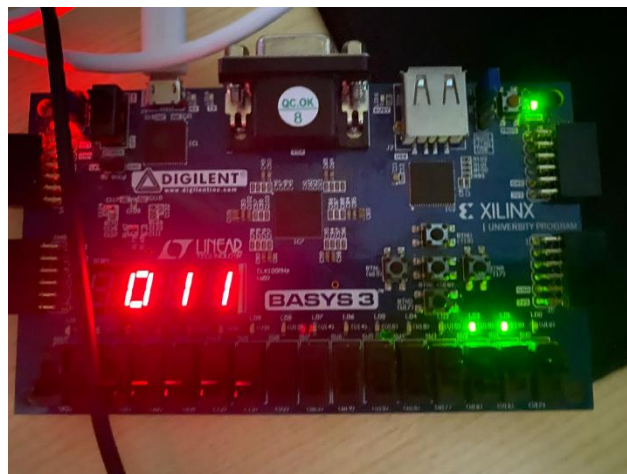
You will have to write the instruction number and what the instruction does, any mistake in Encoding may result in different than expected output. It is recommended to write on A4 or keep track of the instruction, you can use the leds on the board as well.



Continue button will be pressed to advance in the program you can change it to a smaller CLK like a 1sec Clk and it would still work. For program inserted, Fibonnaci you will have to input the n^{th} Fibonnaci Number (in this case its + 2, since 1,1 wasn't counted , 11 is maximum since it will hit 233).

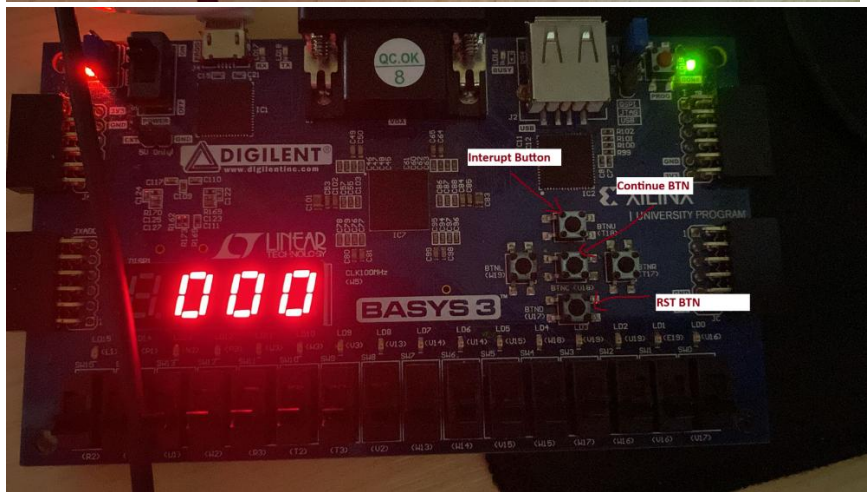
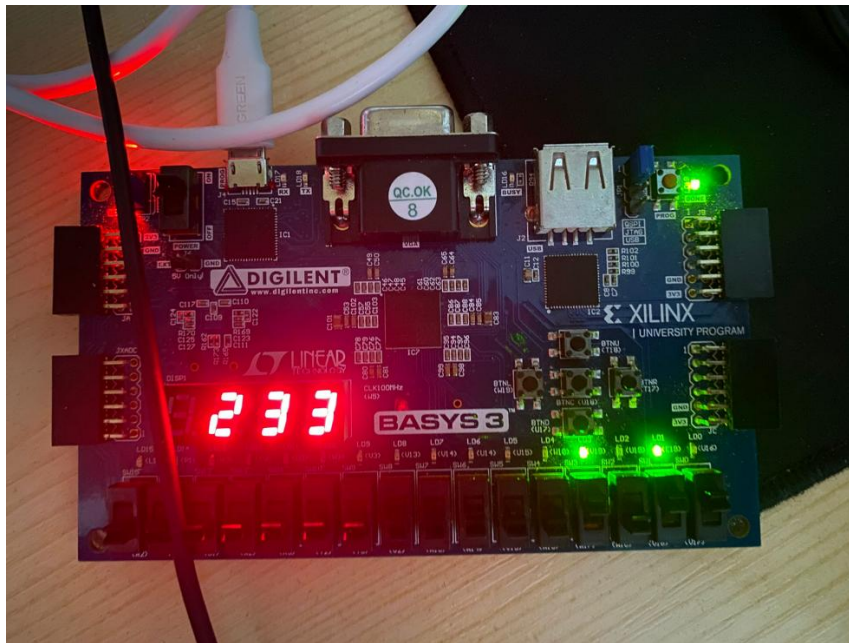


When you will see EEE it means you ve hit an Input instruction now depending on how you wrote the program, the In port will take either the 8 switches from left (if your input port is from 127 – 255) or 8 switches from right. After you put the value you liked the program will continue to execute the next instructions.

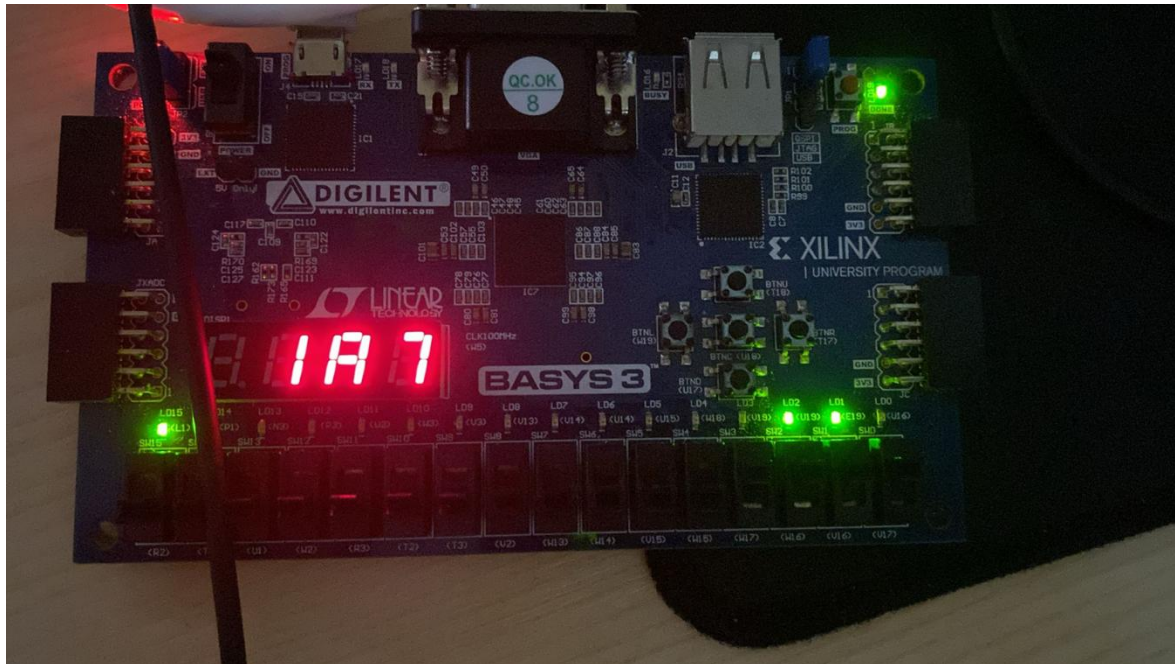


For an example I choose to print the 11 (13 Fibonacci Number which will 233, starting with seq 1,1)

The program will loop through all Fibonacci numbers calculating them by the recurrence formula, 2,3, ... 144, 233. Once it prints 233, it will print again 000 since there is an instruction Output S8, which is Zero. The Fibonacci program load into s0,s1, -> 1 . Uses s2 as sum of these 2 and S10 as a counter. The counter will subtract 1 and then JUMP NOT ZERO back to adding into $s2 = s1 + s0$... After the program finishes successfully it will start again at instruction 0 since there is an unconditional jump to address 0000000, which is instruction 0.



If user presses Interrupt button the next Instruction will have an interrupt for 5 button continues :



The interrupt will not execute the instruction (for the image instruction 4), only after the Interrupt is done, Instruction 4 will be realized.

This project currently support at maximum 256 instruction so take care and write small programs and take into account that you have to deal with overflows in different ways. User is expected to have some coding knowledge before running this on FPGA, projects comes with one more simple program : which inputs 2 numbers from user and prints out the SUM.

The reset button will start the instruction back at 0 or stop the interrupt and start Program Counter back to 0 as well.

CHAPTER 4 : Design Solution Justification

At start of the decoding of each instruction : Write_enable and Update carry and zero flags will be set on 1 since nearly all instructions modify the flags with the exception of Flow control. The Decoder will go through cases, checking first if the instruction is about using a register or a constant. First Bit of the instruction : 0 -> Constant or 1 -> Register.

0 -> Operation code of the instruction and ALU is set at the end of beginning of instruction (14-12) .

1 -> 1100 say we have to work with a register and the operation code is set at the end (3-0).

Then next operation are in the case, Shift left or Shift right we communicate this with a Rotation Signal and Rotation code to ALU-ROTATION.

If those pass then there are the inputs/outputs which flags will be disabled to be modified.

If the cases passes we enter flow control units and we set Update_carry_Zero_flags on 0 and Write_enable on 0 and jump address take the last bits (7-0), now depending on the type of JUMP (bit 12) we set the JUMP Flag accordingly. Next cases are the interrupts.

If there is an interrupt it will allow for interrupts to happen anytime since Interrupt flag is in a register. We use different CLK (Board CLK) in TOP LEVEL PROCESOR since in Interruptor component we want to detect a button press instantly and not wait on the continue btn or press both to get an interrupt. A latch will hold Interrupt_impulse until a Continue Btn is pressed, checks with Interrupt_flag and will execute the Interrupt. If we wouldn't use the Board CLK (5ns) we would have to press simultaneously Continue and Interrupt button.

CHAPTER 5 : Further Development

Implementation of CALL, JUMP, Return functions , STACK. Make UUT for each components.

UART : make a port be able to receive bits from a PC then PicoAssambler do instructions on those bits and sent them back to be printed on PC.

CHAPTER 6 : Bibliography

XAPP213.PDF – reference for implementation of Pico Assambler

Project done by Şandru Sebastian and Cătălin Oltean Marin.

Git hub : [Reorad/8Bit-FPGA-Processor](https://github.com/Reorad/8Bit-FPGA-Processor)